

NIRBHAY GUPTA

90119976

MODULE 3 – ADVANCED SQL QUERIES

Q1.

Five tables require four join conditions, because n tables generally need $n - 1$ join conditions. If only three conditions are supplied, one pair of tables is left without a proper join condition. This creates a Cartesian product, where every row from one table is combined with every row from the other table. The result set becomes much larger and can seriously affect database performance.

Q2.

The INNER JOIN returns only customers who have at least one account. A customer with multiple accounts can appear multiple times.

The LEFT JOIN returns every customer, including customers who do not have an account. For customers without an account, the account columns contain NULL.

For the standard dataset, both queries return 36 rows, because every customer in the relevant data has at least one account. A LEFT JOIN returns at least as many rows as the equivalent INNER JOIN, but not necessarily more.

Q3.

```
SELECT b.branch_name, l.loan_id
FROM branch b
LEFT JOIN loan l
    ON l.branch_id = b.branch_id
    AND l.status = 'OVERDUE'
ORDER BY b.branch_name;
```

Explanation: The condition is moved into the ON clause. This filters the loans that can match while still retaining every branch.

Q4.

An anti-join is generally formed by using an outer join and then selecting only the rows where the join did not find a matching record.

The usual pattern is:

```
SELECT ...
FROM table1 a
LEFT JOIN table2 b
    ON ...
WHERE b.primary_key IS NULL;
```

The column tested for NULL should be the primary key of the second table.

A primary key can never be NULL in an actual row, so NULL means that no matching row was found. A nullable column should not be used because a real matching row could also contain NULL, making it impossible to distinguish a missing match from an existing row with a NULL value.

Q5.

The employee hierarchy query needs a LEFT JOIN because a Branch Manager has manager_id = NULL.

With an INNER JOIN, the Branch Manager needs to have a matching manager record, but NULL does not match any employee. Therefore, Branch Managers would disappear from the result.

In the standard dataset:

- INNER JOIN → **16 rows**
- LEFT JOIN → **24 rows**

The eight missing rows are the eight Branch Managers.

Q6.

Expression	Counts NULLs?	Expected value on account
COUNT(*)	Yes	36
COUNT(interest_rate)	No	36
COUNT(DISTINCT account_type)	No	3
AVG(balance)	No	220372.35

COUNT(*) counts every row, including rows containing NULL values.

COUNT(interest_rate) counts only non-NULL values. There are no NULL values in interest_rate in this dataset, so the result is 36.

COUNT(DISTINCT account_type) returns 3 because the actual data contains SAVINGS, CURRENT and SALARY.

AVG(balance) ignores NULL values and returns 220372.35.

The database constraint permits another account type, NRI, but there are currently no NRI accounts in the data.

Q7.

The GROUP BY rule is:

Every column in the SELECT list must either be included in an aggregate function or appear in the GROUP BY clause.

The database cannot simply select an arbitrary value for an ungrouped column because a group can contain multiple rows with different values.

For example, if a branch contains four accounts with different account numbers, there is no logical basis for choosing one account number to represent the whole branch. Therefore, PostgreSQL rejects the query rather than returning an arbitrary result.

Q8.

	WHERE	HAVING
Filters what?	Individual rows	Groups
Runs when?	Before GROUP BY	After GROUP BY and aggregation
Can reference aggregates?	No	Yes
Which should you prefer?	Prefer WHERE whenever possible	Use when filtering groups/aggregate results

WHERE filters individual rows before grouping takes place.

HAVING filters groups after the GROUP BY and aggregate calculations have been performed.

WHERE should be preferred when the condition can be applied to individual rows because fewer rows need to reach the grouping stage, making the query more efficient.

Q9.

The fixed_deposit table contains NULL values in nominee_name.

Therefore, the subquery returns a list containing NULL.

NOT IN with a list containing NULL results in an unknown comparison. Since WHERE keeps only rows for which the condition is TRUE, no rows pass the condition.

Fix 1 — Exclude NULL values

```
SELECT COUNT(*)
FROM customer c
WHERE c.first_name NOT IN
(
    SELECT nominee_name
    FROM fixed_deposit
    WHERE nominee_name IS NOT NULL
);
```

Fix 2 — Use NOT EXISTS

```
SELECT COUNT(*)
FROM customer c
WHERE NOT EXISTS
(
    SELECT 1
    FROM fixed_deposit f
    WHERE f.nominee_name = c.first_name
);
```

NOT EXISTS is safer because it is not affected by NULL values in the subquery.

Q10.

Fan-out is the multiplication of rows that occurs when one parent row matches multiple child rows during a join.

For example, if one customer has two accounts and those accounts have 21 transactions, joining the customer, account and transaction tables can produce 21 rows for that customer.

A banking report calculating total deposits and transaction value by branch can therefore become incorrect. SUM(a.balance) may count the same account balance once for every transaction belonging to that account.

This produces a believable but inflated result.

The solution is to use appropriate COUNT(DISTINCT ...) expressions or, preferably, aggregate data at separate grains and then join the summaries.

Q11.

```
SELECT b.branch_name,
       c.first_name || ' ' || c.last_name AS customer_name,
       a.account_number,
       k.card_type,
       k.card_number_masked,
       k.status
```

```

FROM card k
JOIN account a
    ON a.account_id = k.account_id
JOIN customer c
    ON c.customer_id = a.customer_id
JOIN branch b
    ON b.branch_id = a.branch_id
ORDER BY b.branch_name, customer_name;
Output: 32 rows, one row per card.
The query uses four tables and three join conditions.

```

Q12.

```

SELECT b.branch_name,
       c.first_name || ' ' || c.last_name AS customer_name,
       a.account_number,
       ROUND(a.balance, 2) AS balance
FROM customer c
JOIN branch b
    ON b.branch_id = c.home_branch_id
LEFT JOIN account a
    ON a.customer_id = c.customer_id
WHERE b.branch_name IN ('Koramangala', 'Whitefield')
ORDER BY a.balance DESC NULLS LAST;

```

The query lists customers from Koramangala and Whitefield, including customers who do not have an account.

The LEFT JOIN is used for the account because the account is optional.

The branch condition is placed in WHERE because the branch must match.

Q13.

```

SELECT a.account_number,
       a.account_type,
       a.status,
       c.first_name || ' ' || c.last_name AS customer_name
FROM account a
JOIN customer c
    ON c.customer_id = a.customer_id
LEFT JOIN account_txn t
    ON t.account_id = a.account_id
WHERE t.txn_id IS NULL;

```

Output

account_number	account_type	status	customer_name
ICIC0000009001	CURRENT	DORMANT	Aarav Sharma

Row count: 1

The LEFT JOIN allows accounts without transactions to remain in the result, and t.txn_id IS NULL identifies those accounts.

Q14.

```

SELECT a.account_number,
       a.account_type,
       c.first_name || ' ' || c.last_name AS customer_name
FROM account a
JOIN customer c
      ON c.customer_id = a.customer_id
LEFT JOIN card k
      ON k.account_id = a.account_id
WHERE k.card_id IS NULL
ORDER BY a.account_number;

```

Output

account_number	account_type	customer_name
ICIC0000009033	SAVINGS	Pooja Rao
ICIC0000009034	CURRENT	Suresh Singh
ICIC0000009035	SALARY	Anjali Verma
ICIC0000009036	SAVINGS	Anjali Verma

Count: 4

These are accounts without an issued card.

Q15.

```

SELECT e.first_name || ' ' || e.last_name AS employee,
       e.designation,
       b.branch_name,
       COALESCE(
         m.first_name || ' ' || m.last_name,
         '(no manager)'
       ) AS reports_to
FROM employee e
JOIN branch b
      ON b.branch_id = e.branch_id
LEFT JOIN employee m
      ON m.employee_id = e.manager_id
ORDER BY b.branch_name, e.employee_id;

```

Output: 24 rows

LEFT JOIN is required because Branch Managers do not have a manager.
COALESCE displays (no manager) instead of leaving the value NULL.

Q16.

```

SELECT m.first_name || ' ' || m.last_name AS manager,
       b.branch_name,
       COUNT(e.employee_id) AS team_size,
       ROUND(SUM(e.salary), 2) AS team_salary
FROM employee m
JOIN branch b
      ON b.branch_id = m.branch_id
LEFT JOIN employee e
      ON e.manager_id = m.employee_id

```

```
WHERE m.designation = 'Branch Manager'
GROUP BY m.employee_id,
         m.first_name,
         m.last_name,
         b.branch_name
ORDER BY b.branch_name;
```

Output: 8 rows

Each Branch Manager has a team size of **2**.

The manager's own salary is not included because only employees whose manager_id points to that manager are included.

Q17.

Step 1 — Create the table

```
CREATE TABLE loan_risk_band
(
    band_code VARCHAR(4) NOT NULL,
    band_name VARCHAR(20) NOT NULL,
    min_outstanding NUMERIC(15,2) NOT NULL,
    max_outstanding NUMERIC(15,2) NOT NULL,
    CONSTRAINT pk_loan_risk_band PRIMARY KEY (band_code)
);
```

Step 2 — Insert the bands

```
INSERT INTO loan_risk_band
VALUES
('L1', 'Small', 0.00, 500000.00),
('L2', 'Medium', 500000.00, 2000000.00),
('L3', 'Large', 2000000.00, 5000000.00),
('L4', 'Very Large', 5000000.00, 99999999.00);
```

Step 3 — Non-equi join

```
SELECT r.band_name,
       COUNT(*) AS loans,
       ROUND(SUM(l.outstanding_amount), 2) AS total_outstanding
FROM loan l
JOIN loan_risk_band r
    ON l.outstanding_amount >= r.min_outstanding
   AND l.outstanding_amount < r.max_outstanding
GROUP BY r.band_code, r.band_name
ORDER BY r.band_code;
```

The ranges are **half-open**: $\geq$ minimum and $<$ maximum. This prevents a loan exactly on a boundary from being counted in two bands.

Q18.

```
SELECT b.branch_name,
       COUNT(DISTINCT c.customer_id) AS customers,
       COUNT(a.account_id) AS accounts,
       ROUND(COALESCE(SUM(a.balance), 0), 2) AS total_balance,
       ROUND(AVG(a.balance), 2) AS avg_balance
```

```

FROM branch b
LEFT JOIN customer c
    ON c.home_branch_id = b.branch_id
LEFT JOIN account a
    ON a.customer_id = c.customer_id
GROUP BY b.branch_name
ORDER BY total_balance DESC;

```

Explanation

COUNT(DISTINCT c.customer_id) prevents customers with multiple accounts from being counted multiple times.

COUNT(a.account_id) gives zero for a branch with no accounts.

LEFT JOIN ensures that branches without customers/accounts are not removed.

Q19.

```

SELECT l.loan_type,
    COUNT(*) AS loans,
    ROUND(SUM(l.principal_amount), 2) AS principal,
    ROUND(SUM(l.outstanding_amount), 2) AS outstanding,
    ROUND(
        100.0 * SUM(l.outstanding_amount)
        / SUM(l.principal_amount),
        1
    ) AS pct_outstanding
FROM loan l
GROUP BY l.loan_type
ORDER BY outstanding DESC;

```

Output

loan_type	loans	principal	outstanding	pct_outstanding
HOME	3	6200000.00	5558681.93	89.7
GOLD	2	6080000.00	3254891.93	53.5
PERSONAL	2	3450000.00	2751096.02	79.7
AUTO	3	2250000.00	1733612.66	77.0
EDUCATION	2	1750000.00	1319520.49	75.4

The percentage is calculated using:

$\text{SUM}(\text{outstanding}) / \text{SUM}(\text{principal}) \times 100$

rather than averaging individual loan percentages.

Q20.

```

SELECT t.channel,
    SUM(CASE
        WHEN t.status = 'SUCCESS' THEN 1
        ELSE 0
    END) AS successful,
    SUM(CASE
        WHEN t.status = 'FAILED' THEN 1
        ELSE 0
    END) AS failed,

```

```

ROUND(
    SUM(CASE
        WHEN t.status = 'SUCCESS' THEN t.amount
        ELSE 0
    END),
    2
) AS success_value,
ROUND(
    100.0 *
    SUM(CASE
        WHEN t.status = 'FAILED' THEN 1
        ELSE 0
    END)
    / COUNT(*),
    2
) AS failure_rate
FROM account_txn t
GROUP BY t.channel
ORDER BY failure_rate DESC;

```

Output

channel	successful	failed	failure_rate
POS	56	4	6.67
NEFT	61	4	6.15
IMPS	52	3	5.45
ONLINE	57	2	3.39
ATM	61	2	3.17
BRANCH	57	1	1.72
UPI	64	0	0.00
RTGS	71	0	0.00

Total failures: 16.

Q21.

```

SELECT b.branch_name,
    COUNT(*) AS loans,
    ROUND(SUM(l.outstanding_amount), 2) AS total_outstanding
FROM loan l
JOIN branch b
    ON b.branch_id = l.branch_id
GROUP BY b.branch_name
HAVING SUM(l.outstanding_amount) > 2000000
ORDER BY total_outstanding DESC;

```

Output

branch_name	loans	total_outstanding
T Nagar Chennai	3	6215460.74
Koramangala	3	4780879.96
Fort Mumbai Main	3	2655077.80

Count: 3 branches.

Q22.

```

SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer_name,
       COUNT(*) AS accounts,
       ROUND(SUM(a.balance), 2) AS combined_balance
FROM customer c
JOIN account a
  ON a.customer_id = c.customer_id
GROUP BY c.customer_id,
         c.first_name,
         c.last_name
HAVING COUNT(*) >= 2
ORDER BY combined_balance DESC;

```

Output

customer_id	customer_name	accounts	combined_balance
1020	Ritu Banerjee	2	654237.30
1030	Anjali Verma	2	587663.55
1010	Priya Verma	2	453005.40
1025	Farhan Gupta	2	437520.63
1015	Imran Bose	2	187104.39
1005	Karthik Gupta	2	119366.88

Count: 6 customers.**Q23.**

```

SELECT TO_CHAR(t.txn_date, 'YYYY-MM') AS month,
       COUNT(*) AS txns,
       ROUND(
         SUM(CASE
              WHEN t.txn_type = 'CREDIT' THEN t.amount
              ELSE 0
            END),
         2
       ) AS credits,
       ROUND(
         SUM(CASE
              WHEN t.txn_type = 'DEBIT' THEN t.amount
              ELSE 0
            END),
         2
       ) AS debits,
       ROUND(
         SUM(CASE
              WHEN t.txn_type = 'CREDIT'
                THEN t.amount
              ELSE -t.amount
            END),

```

2

```
) AS net
FROM account_txn t
WHERE t.status = 'SUCCESS'
GROUP BY TO_CHAR(t.txn_date, 'YYYY-MM')
ORDER BY month;
```

Output

month	txns	credits	debits	net
2026-01	176	3441051.86	5206131.14	-1765079.28
2026-02	135	3194498.15	4911830.04	-1717331.89
2026-03	168	3724339.24	5117754.77	-1393415.53

The query considers only successful transactions.

Q24.

```
SELECT account_number,
       ROUND(balance, 2) AS balance
FROM account
WHERE balance > 2 * (
    SELECT AVG(balance)
    FROM account
)
ORDER BY balance DESC;
```

Output

account_number	balance
ICIC0000009018	473345.84

The bank-wide average balance is 220372.35.

Twice the average is:

440744.70

Therefore, only the account with balance 473345.84 qualifies.

Q25.

```
SELECT b.branch_name,
       (
           SELECT ROUND(MAX(a.balance), 2)
           FROM account a
           WHERE a.branch_id = b.branch_id
       ) AS largest_account
FROM branch b
ORDER BY largest_account DESC;
```

Output

branch_name	largest_account
Connaught Place	473345.84
Andheri East	439665.56
Whitefield	421425.91
Koramangala	410879.83
Fort Mumbai Main	408783.99
Banjara Hills	400457.72

T Nagar Chennai | 276393.25

Baner Pune | 119212.62

The subquery is correlated because it uses b.branch_id from the outer query.

Q26.

Using NOT EXISTS

```
SELECT c.customer_id,  
       c.first_name,  
       c.last_name  
FROM customer c  
WHERE EXISTS (  
    SELECT 1  
    FROM loan l  
    WHERE l.customer_id = c.customer_id  
)  
AND NOT EXISTS (  
    SELECT 1  
    FROM fixed_deposit f  
    WHERE f.customer_id = c.customer_id  
)  
ORDER BY c.customer_id;
```

Using LEFT JOIN anti-join

```
SELECT DISTINCT  
       c.customer_id,  
       c.first_name,  
       c.last_name  
FROM customer c  
JOIN loan l  
    ON l.customer_id = c.customer_id  
LEFT JOIN fixed_deposit f  
    ON f.customer_id = c.customer_id  
WHERE f.deposit_id IS NULL  
ORDER BY c.customer_id;
```

Output

Both queries return the same **5 customers**:

1009

1011

1013

1017

1019

DISTINCT is required in the second query because a customer may have multiple loans.

Q27.

```
SELECT ct.customer_id,  
       ct.customer_name,  
       ct.total_value  
FROM
```

```

(
  SELECT c.customer_id,
         c.first_name || ' ' || c.last_name AS customer_name,
         SUM(t.amount) AS total_value
  FROM customer c
  JOIN account a
    ON a.customer_id = c.customer_id
  JOIN account_txn t
    ON t.account_id = a.account_id
  WHERE t.status = 'SUCCESS'
  GROUP BY c.customer_id,
           c.first_name,
           c.last_name
) AS ct
WHERE ct.total_value >
(
  SELECT AVG(total_value)
  FROM
  (
    SELECT SUM(t.amount) AS total_value
    FROM account_txn t
    JOIN account a
      ON a.account_id = t.account_id
    WHERE t.status = 'SUCCESS'
    GROUP BY a.customer_id
  ) AS per_customer
)
ORDER BY ct.total_value DESC;

```

Explanation

The inner query calculates the total successful transaction value for each customer. The outer query compares each customer's total with the average customer total. Only customers whose total is above the overall average are returned.

Q28.

```

SELECT customer_id,
       product_type,
       ROUND(amount, 2) AS amount
FROM
(
  SELECT customer_id,
         'DEPOSIT' AS product_type,
         principal_amount AS amount
  FROM fixed_deposit

  UNION ALL

  SELECT customer_id,

```

```
        'LOAN' AS product_type,  
        outstanding_amount AS amount  
FROM loan
```

UNION ALL

```
SELECT customer_id,  
        'ACCOUNT' AS product_type,  
        balance AS amount  
FROM account  
) AS holdings  
ORDER BY customer_id, product_type;
```

Output

68 rows

- 20 fixed deposits
- 12 loans
- 36 accounts

UNION ALL is used because different product holdings should not be removed merely because they happen to have identical amounts.

Q29.

1. Customers holding both a deposit and a loan

```
SELECT customer_id  
FROM fixed_deposit
```

INTERSECT

```
SELECT customer_id  
FROM loan;
```

Count: 7

2. Customers holding a deposit but no loan

```
SELECT customer_id  
FROM fixed_deposit
```

EXCEPT

```
SELECT customer_id  
FROM loan;
```

Count: 13

3. Customers holding a loan but no deposit

```
SELECT customer_id  
FROM loan
```

EXCEPT

```
SELECT customer_id
FROM fixed_deposit;
Count: 5
```

4. Customers holding at least one of the two products

```
SELECT customer_id
FROM loan
```

UNION

```
SELECT customer_id
FROM fixed_deposit;
```

Count: 25

Consistency check

Both deposit + loan = 7

Deposit only = 13

Loan only = 5

Total = 25

Therefore:

7 + 13 + 5 = 25

which matches the UNION result.

Also:

- Deposit holders = 7 + 13 = **20**
- Loan holders = 7 + 5 = **12**
- Customers with neither product = 30 - 25 = **5**

Q30.

Defects in the original query

There are **four defects**:

1. COUNT(c.customer_id) is inflated because transactions create multiple rows for the same customer.
2. SUM(a.balance) is also inflated because an account balance is repeated for every transaction.
3. BETWEEN DATE '2026-01-01' AND DATE '2026-03-31' misses transactions later on March 31 because txn_date is a timestamp.
4. The inner joins remove branches with no transactions, and failed transactions are included because there is no status = 'SUCCESS' filter.

Correct query

WITH deposits AS

```
(
  SELECT c.home_branch_id AS branch_id,
         COUNT(DISTINCT c.customer_id) AS customers,
         COUNT(a.account_id) AS accounts,
         SUM(a.balance) AS total_deposits
  FROM customer c
  LEFT JOIN account a
    ON a.customer_id = c.customer_id
  GROUP BY c.home_branch_id
```

```

),
txns AS
(
    SELECT a.branch_id,
           SUM(t.amount) AS total_transacted
    FROM account_txn t
    JOIN account a
        ON a.account_id = t.account_id
    WHERE t.txn_date >= DATE '2026-01-01'
    AND t.txn_date < DATE '2026-04-01'
    AND t.status = 'SUCCESS'
    GROUP BY a.branch_id
)
SELECT b.branch_name,
       COALESCE(d.customers, 0) AS customers,
       COALESCE(d.accounts, 0) AS accounts,
       ROUND(COALESCE(d.total_deposits, 0), 2) AS total_deposits,
       ROUND(COALESCE(x.total_transacted, 0), 2) AS total_transacted
FROM branch b
LEFT JOIN deposits d
    ON d.branch_id = b.branch_id
LEFT JOIN txns x
    ON x.branch_id = b.branch_id
ORDER BY total_deposits DESC;

```

The corrected version calculates deposits and transactions separately, avoiding fan-out before joining the results.

Q31.

Answer:

GROUP BY produces one row for each group. However, "top three customers per branch" requires several rows within each branch to be ranked.

HAVING can only filter complete groups; it cannot rank individual customers inside a group.

A **window function** is required, such as:

```

ROW_NUMBER() OVER (
    PARTITION BY branch_id
    ORDER BY balance DESC
)

```

The rows can then be filtered to keep the top three.

Q32.

A report whose correct result is normally zero provides no obvious warning when it stops working. In this case, six weeks of zero exceptions could actually mean that the report was broken rather than that there were no compliance issues.

The report should therefore include a positive control, such as the number of records examined along with the number of exceptions found.

For example, 0 exceptions from 412,000 transactions checked is different from 0 exceptions from 0 transactions checked.

A known test record or an alert when the population count unexpectedly falls can provide an additional check.

NOT EXISTS should also be preferred over NOT IN where appropriate because it avoids the NULL-related failure demonstrated earlier.

Q33

Without changing the SQL, I would take these three actions:

1. Add appropriate indexes

Create indexes for the access paths already used by the query, especially:

account_txn(account_id, txn_date)

and relevant foreign-key columns.

2. Move the report to a read replica

Run the report against a read replica rather than the transactional primary database so that the large scan does not compete with branch-opening activity for resources.

3. Materialise the report overnight

Run the report before the branches open and store its results in a summary table. The 9 a.m. report can then read the pre-computed results instead of scanning 40 million transaction rows.

Other possible approaches include Redis caching and monthly partitioning of account_txn.

Q34.

First, extract the same columns from both systems into two tables with identical structures.

Transactions in acquired bank but missing from ICICI

```
SELECT txn_id,
       account_id,
       txn_date,
       amount
FROM acquired_extract
EXCEPT
SELECT txn_id,
       account_id,
       txn_date,
       amount
FROM icici_core_txn;
```

A non-empty result means transactions present in the acquired-bank extract did not appear in ICICI's core system. Possible reasons include a failed load, rejected rows or an incorrect filter.

Transactions in ICICI but not in acquired-bank extract

```
SELECT txn_id,
       account_id,
       txn_date,
       amount
FROM icici_core_txn
EXCEPT
SELECT txn_id,
       account_id,
       txn_date,
       amount
FROM acquired_extract;
```

A non-empty result means ICICI contains transactions that were not present in the source extract. Possible reasons include duplicate loading, an incorrect date range or an incorrect join during migration.

Q35.

Ticket ICICI-4530

Subject: Branch Performance Pack – Summary

Dear Ms. Rao,

The branch performance pack now provides a consolidated view of branch performance from the core banking data.

It includes branch summaries, dormant customers, large transactions, deposits approaching maturity, and product holdings.

One finding is that T Nagar Chennai recorded 2,691,432 compared with 347,122 for Whitefield. This represents a substantial difference between the two branches despite comparable customer numbers.

Another finding is that seven of the eight branches paid out more than they received during the quarter.

Banjara Hills was the only branch close to being balanced.

These results may require further investigation before operational conclusions are made.

One limitation is that transactions are attributed to the customer's home branch rather than the physical branch where the transaction occurred.

For example, a customer from Fort Mumbai making a withdrawal in Pune is still counted against Fort Mumbai.

Therefore, the report should not be used directly to measure branch footfall or physical transaction activity.

Additional transaction-level branch information would be required for that purpose.